

우를 찾을 수 있습니다. 이때의 문자열의 길이가 flag의 길이의 딱 2배가 되는 순간입니다. 이를 통해 flag에 해당하는 문자열의 길이를 알아낸 후 해당 길이의 문자열에 해당 문자를 대입하여 브루트 포스 기법을 사용하면 flag 값을 알아낼 수 있습니다.

2.3 풀이 결과

해당 파이썬 코드를 통해 브루트 포스를 시행하여 flag 값을 추정하면 아래와 같은 flag를 확인 할 수 있습니다.

```
flag{byte_to_byte_leak}
```

3.1.1

III. 풀이과정 중 학습한 내용

3.1 Hash 함수

Hash 함수는 입력 받은 값의 크기와 상관없이 고정된 길이의 결과 값을 출력하는 함수입니다. 이 중에서도 보안적 해시 함수와 비보안적 해시 함수가 나뉘는데 이를 나누는 특성은 다음과 같습니다.

첫째로 해당 hash함수의 수학적 계산법이 역연산 불가능해야 합니다. 이를 통해 해시 값이 공개적이라도 이를 통해 입력을 유추 할 수 없게 해야 하기 때문입니다.

둘째로 입력값이 하나라도 다르다면 해시값이 매우 큰 차이가 나야 하는 일명 눈사태 효과가 발생해야 합니다. 그렇지 않다면 비슷한 입력에서 비슷한 해시값이 도출되어 원래의 입력 값을 추정하기 쉬워지기 때문입니다.

마지막으로 서로 다른 입력에 대해 같은 값을 내는 가능성이 아주 희박해야 합니다. 만약 다른 입력도 해시값이 동일하다면 이는 원래 들어와야 할 입력과 달라도 이를 판단하기 어렵기 때문입니다.

3.1.1 md5 해시 함수

현재 문제에서 사용된 해시 함수는 md5라는 해시 함수입니다. 이 함수는 보안성이 있다는 해시 함수라 과거에 여겨졌으나 현재엔 결함이 확인되어 암

호화나 보안적으로는 사용되지 않는 함수입니다. 해당 해시 함수의 계산같은 것은 풀이와 큰 상관이 없기에 이 함수에서 발견된 결함에 대해서 말하려 합니다.

첫째로 해당 해시 함수의 계산 속도가 빠르다는 점입니다. 이는 앞에서 제시한 방식인 브루트 포스를 하기에 시간적 부담이 없다는 점에서 결함입니다.

다음으로 해당 해시값이 128bit라는 점입니다. 이는 2의 128승의 해시값을 가질 수 있으나 현대의 컴퓨터의 계산 속도라면 이를 확인하는 작업이 불가능하지 않다는 것 또한 현재는 사용하지 않는 이유 중 하나입니다.

마지막으로 해당 함수는 수학적으로 다른 입력에도 값을 조절하여 같은 해시값을 낼 수 있는 이른바 충돌 생성이 가능합니다. 해당 해시 함수는 차분 공격이라는 역산 방식을 사용하여 다른 입력에도 동일한 결과를 도출할 수 있는 상태입니다.

3.1.1 3.1.2 차분 공격 방식

Md5해시 함수의 경우 512비트 블록을 입력으로 받아 4개의 32비트 레지스터(A, B, C, D)를 여러 라운드에 걸쳐 반복적으로 변환하는 구조입니다. 이는 입력 블록간의 차이만 동일하게 조정하면 해시 값이 동일하게 만들 수 있습니다. 이 방식을 사용하면 다른 문서임에도 동일한 해시 함수를 가지기에 문서 인증의 효력에 공신력이 없게 만듭니다. 이런 점 때문에 현재 md5 해시 함수는 보안적 해시 함수로써 사용이 중단되었습니다.

3.1.3 다른 해시 함수와 비교

Md5와 비교했을 때 현재 보안적으로 사용중인 해시함수에 대해서 알아보려 합니다.

첫번째로 해시값의 범위가 다릅니다. 해시값의 범위가 128bit에 불과한 MD5와 달리 현재 보안적으로 사용중인 SHA-256같은 경우 256bit에 해당하기 때문에 현재의 컴퓨터라도 이를 전부 계산하는 것은 불가능에 가깝습니다.

두번째로 MD5와 같이 수학적으로 다른 무서라도 같은 해시 값을 가지게 만들 수 있는 방법이 현재로서는 전무합니다.

이런 점들 때문에 현재 MD5는 보안적인 해시 함수로는 상용이 되지 않습니다.

IV. 결론

해당 실습을 통해 해시 함수의 개념과 보안성이 있는 해시 함수와 그렇지 않은 것의 차이를 알고 이를 구분하여 사용해야 합니다. 그리고 해시 함수가 아무리 뛰어나도 이를 활용한 웹페이지나 앱의 설계에서 실수를 한다면 공격에서 자유로울 수 없기에 암호화 방식 뿐만 아니라 앱과 웹의 설계에서도 보안적 결함을 생각해야 합니다. 마지막으로 컴퓨터의 계산 능력의 발달이나 수학적 계산 공식의 발전으로도 보안적 결점이 발생 할 수 있기에 보안은 계속해서 공부하고 발전시켜야 합니다.